Deliverable 2: Database & CRUD

1. Introductie en sprintdoel

De Garden Group Incident Management applicatie ondersteunt de IT-servicedesk bij het registreren en verwerken van incidenten en het beheren van gebruikers. Het doel van deze sprint (Deliverable 2) is het realiseren van de MongoDB CRUD-operaties (Create, Read, Update, Delete) voor zowel tickets als medewerkers, evenals het ontwikkelen van een dashboard dat inzicht biedt in de huidige status van tickets en de werkdruk. Dit document beschrijft de gezamenlijke architectuur, database keuzes en de specifieke implementaties van de Tickets-module, Employees-module en Dashboard-module.

2. Database en technologiekeuze

De applicatie is gebouwd in C# (ASP.NET MVC) en maakt gebruik van een MongoDB database, verbonden via de officiële MongoDB.Driver NuGet-package. MongoDB is gekozen vanwege het flexibele documentmodel, wat ideaal is voor incidentbeheer en snelle data-opslag.

Documentstructuur en Relaties:

- **Tickets Collectie:** Tickets worden opgeslagen in een eigen collectie. Er is overwogen om tickets in te sluiten (embedding) in het Employee document, maar dit zou ertoe kunnen leiden dat de array ongecontroleerd groeit. Omdat de Service Desk tickets vaak onafhankelijk van de medewerker moet kunnen inzien, sluit een aparte collectie met referenties beter aan bij de toegang patronen.
- **Employees Collectie:** Bevat medewerkers gegevens (zoals naam, email, telefoonnummer en rol) in een flexibel JSON-formaat.
- **Dashboard Modellen:** Voor de dashboard statistieken worden de resultaten direct vanuit MongoDB gemapt naar C# typed model objects (Status Percentage and EmployeeWorkload). Met [BetonElement("...")] attributen is de koppeling tussen afwijkende MongoDB-veldnamen en C#-properties expliciet gemaakt.
- **Connectie:** De connectionstring en database configuratie worden veilig opgehaald via een afzonderlijke Secret Service en .NET User Secrets. Hierdoor staan er geen wachtwoorden in de broncode of appsettings.json.
- **Indexing:** Alle indexen op tickets-en employees-collectie worden één keer bij het opstarten van de applicatie aangemaakt door Mongo Index Initializer.Ensure Indexes (aangeroepen in Program.cs).

3. CRUD-implementatie (Tickets & Employees)

Tickets Module

Deze module beheert de incidenten.

- **Create (Aanmaken):** Slaat een nieuw incident op in de database.
    - **Methode:** `TicketRepository.CreateTicket(Ticket ticket)`

```
public void CreateTicket(Ticket ticket)
{
    _ticketCollection.InsertOne(ticket);
}
```

Read (Ophalen): Haalt tickets uit de database voor de overzichten.

- **Methode:** `TicketRepository.GetAllTickets()` / `TicketRepository.GetTicketById(ObjectId id)`

```
public List<Ticket> GetAllTickets()
{
    return _ticketCollection.Find(t => true).ToList();
}

public Ticket GetTicketById(ObjectId id)
{
    return _ticketCollection.Find(t => t.Id == id).FirstOrDefault();
}
```

- **Update (Bewerken):** Wijzigt details of statussen van een bestaand ticket in de database.
    - **Methode:** `TicketRepository.UpdateTicket(Ticket ticket)`

```
public void UpdateTicket(Ticket ticket)
        {
            var filter = Builders<Ticket>.Filter.Eq(t => t.Id, ticket.Id);

            var update = Builders<Ticket>.Update
                .Set(t => t.Title, ticket.Title)
                .Set(t => t.Description, ticket.Description)
                .Set(t => t.AssignedToId, ticket.AssignedToId)
```

```
                .Set(t => t.Priority, ticket.Priority)
                .Set(t => t.Status, ticket.Status)
                .Set(t => t.UpdatedAt, DateTime.Now);

            if (ticket.Status == Statuses.Resolved && ticket.ResolvedAt == null)
            {
                update = update.Set(t => t.ResolvedAt, DateTime.Now);
            }
            if (ticket.Status == Statuses.Closed && ticket.ClosedAt == null)
            {
                update = update.Set(t => t.ClosedAt, DateTime.Now);
            }

            _tickets.UpdateOne(filter, update);
        }
```

- **Delete (Verwijderen):** Verwijder onjuiste of dubbele tickets uit de database.
    - **Methode:** `TicketRepository.DeleteTicket(string id)`

```
public void DeleteTicket(string id)
{
    var filter = Builders<Ticket>.Filter.Eq(t => t.Id, id);
    _ticketCollection.DeleteOne(filter);
}
```

Create (Aanmaken)

Voegt een nieuwe medewerker toe aan de database bij onboarding.

- **Klasse/Methode:** `EmployeeDao.Insert(Employee employee)` / `EmployeeService.CreateEmployee(...)`

```
public void CreateEmployee(Employee employee)
{
    _employeeCollection.InsertOne(employee);
}
```

Read (Ophalen)
Haalt alle medewerkers op voor het overzicht of zoekt een specifieke gebruiker op e-mailadres voor authenticatie.

- **Klasse/Methode:** `EmployeeService.GetAllEmployees()` / `EmployeeService.GetEmployeeByEmail(string email)`

```
public List<Employee> GetAllEmployees()
{
    return _employeeCollection.Find(e => true).ToList();
}

public Employee GetEmployeeByEmail(string email)
{
    return _employeeCollection.Find(e => e.Email == email).FirstOrDefault();
}
```

Update (Bewerken)
Wijzigt bestaande gegevens van een medewerker (zoals telefoonnummer, rol of locatie).

- **Klasse/Methode:** `EmployeeService.UpdateEmployee(Employee employee)`

```
public void UpdateEmployee(Employee employee)
{
    var filter = Builders<Employee>.Filter.Eq(e => e.Id, employee.Id);
    _employeeCollection.ReplaceOne(filter, employee);
}
```

Delete (Verwijderen)
Verwijder het profiel van een medewerker die de organisatie verlaat.

- **Klasse/Methode:** `EmployeeService.DeleteEmployee(ObjectId id)`

```
public void DeleteEmployee(ObjectId id)
{
    var filter = Builders<Employee>.Filter.Eq(e => e.Id, id);
    _employeeCollection.DeleteOne(filter);
}
```

4. Queries en aggregaties

Voor het zoeken en filteren van medewerkers (zoals zoeken op e-mail of ID) zijn directe MongoDB-filters en LINQ-expressies gebruikt. queries bestaan voornamelijk uit enkelvoudige document-lookups en eenvoudige overzichtslijsten.

De lijstquery voor tickets combineert filter, sortering en projectie in één `Find`. De volgende indexen ondersteunen die queries en worden aangemaakt in `MongoIndexInitializer`:

Index	Velden	Query waarvoor
`ix_status_createdAt`	status, createdAt	Tickets filteren op status, nieuwste eerst
`ix_assignedTo_createdAt`	assignedTo, createdAt	Tickets van één medewerker, nieuwste eerst
`createdAt_-1`	createdAt	Alle tickets op datum sorteren
`ix_reportedBy_status`	reportedBy, status	Dashboard: statuspercentages per melder
`ux_ticketId` (uniek)	ticketId	Unieke ticketnummers en snel het hoogste nummer ophalen

5. Alternatieven en afwegingen

Tickets Crud

Tijdens de ontwikkeling is er nagedacht over de manier waarop de applicatie communiceert met MongoDB. Hierbij is de afweging gemaakt tussen de huidige synchrone aanpak en een asynchroon alternatief.

Het Alternatief: Asynchrone methodes (`async` / `await`) In plaats van de huidige synchrone methodes zoals `ToList()`, `InsertOne()` en `ReplaceOne()`, biedt de MongoDB C# driver asynchrone alternatieven, zoals `ToListAsync()`, `InsertOneAsync()` en `ReplaceOneAsync()`. Dit betekent dat de database-aanroepen het sleutelwoord `await` krijgen en dat de functies een `Task` of `Task<T>` retourneren.

Voordelen van het asynchrone alternatief:

- **Betere schaalbaarheid:** Wanneer de applicatie een verzoek naar de database stuurt (een I/O-operatie), hoeft de server-thread niet te wachten tot het antwoord terug is. De thread komt vrij om andere web verzoeken af te handelen.
- **Performance:** Bij een applicatie met veel gelijktijdige gebruikers voorkomt dit een bottleneck en blijft de API veel responsiever.

Nadelen en de keuze voor de huidige (synchrone) opzet:

- **Complexiteit:** Asynchrone code werkt besmettelijk door de hele applicatie (ook wel "async all the way" genoemd). Als de *Repository* asynchroon wordt, moeten ook de *Services* en de *Controllers* asynchroon worden gemaakt.
- **Reden van huidige keuze:** Voor de huidige scope van dit project (waarbij de focus ligt op een werkende CRUD-architectuur en overzichtelijke code) is er doelbewust gekozen voor synchrone aanroepen. Dit houdt de code simpeler, beter leesbaar en makkelijker te debuggen.

Mocht de applicatie in de toekomst uitgroeien tot een groot platform met veel dataverkeer, dan is de overstap naar `async/await` het belangrijkste alternatief om door te voeren.

Employee Crud

- **Alternatief:** Alle medewerkers bij het opstarten in een C# `List<Employee>` laden en lokaal filteren met LINQ.
- **Gekozen oplossing:** Direct filteren en zoeken op de MongoDB-database met query-filters (`Builders<Employee>.Filter`).
- **Afweging:** Server-side filteren voorkomt onnodig netwerkverkeer en hoog geheugengebruik in de applicatie zodra het aantal medewerkers groeit.

Tickets en indexen

Alternatief	Gekozen oplossing	Afweging
Losse enkelvoudige indexen op `status` en `assignedTo`	Samengestelde indexen met `createdAt` als tweede veld	Filteren en sorteren worden door één index afgehandeld. De enkelvoudige indexen waren overbodig en kosten alleen snelheid.
Indexen aanmaken in de `TicketRepository`-constructor	Eenmalig bij het opstarten in `Program.cs`	De repository is `Scoped` en wordt per request aangemaakt, dus de constructor zou bij elk request indexen aanmaken
Drie aparte methodes (op status, op medewerker, op datum)	Eén `GetTickets` met optionele filters	Filters zijn combineerbaar en er is geen dubbele code.
Alle tickets laden en het hoogste `TicketId` in C# bepalen	`GetLastTicketId` via de unieke index, met retry bij duplicate key	Constante kosten per insert en veilig bij gelijktijdig aanmaken, nadeel is een mogelijke herhaalpoging.
Een aparte teller-collectie voor ticketnummers	Retry op de unieke index	Geen extra collectie nodig.Een teller is beter bij zeer veel gelijktijdige tickets.

Volledige tickets in de lijst ophalen	`description` weglaten met een projectie	Minder data per lijst verzoek. De beschrijving wordt pas op de detailpagina geladen
Tickets zonder behandelaar meetellen in de werkverdeling	`Niet meetellen`	Een werkverdeling per medewerker heeft geen betekenis voor onbehandelde tickets.
`$facet` of `$set Window Fields` (MongoDB 5.0+) in plaats van meerdere `$group`-stappen	`Losse $group-stappen`	De alternatieven kunnen korter zijn, maar zijn lastiger te lezen. We kozen voor duidelijke stappen die we in Compass één voor één konden controleren.

Gerelateerde medewerker ophalen

Beslissing: $lookup in MongoDB. Alternatief: het ticket ophalen en daarna reporter en assignee met twee extra Find-queries in C#. Afweging: drie queries zijn simpeler te lezen, de pipeline is één round trip maar vraagt handmatig mappen (GetFirstEmployee). Onderbouwing: door het verwijzingsmodel is dit het enige scherm dat beide kanten toont, dus hier is de $lookup echt nodig. De lijstweergave gebruikt bewust geen $lookup.

6. Werkverdeling

Teamlid	Geïmplementeerde functionaliteit	Relevante bestanden / Klassen
Kyano	Complete Employees-module: datamodel, CRUD-operaties, DAO-/servicelaag en integratie met de gebruikersinterface.	Employee.cs, IEmployeeRepository.cs, IEmployeeService.cs, EmployeeRepository.cs, EmployeeService.cs

Frank	$lookup, errorhandling en integratie: bij ticket de reporter en assignee tonen, ongeldige ticket en employee-ids een nette error geven, ontbrekende tickets/medewerkers geven 404 en duplicaten voorkomen.	TicketRepository.GetTicketWithEmployees, TicketDetails, Views/Ticket/Details.cshtml ObjectId-controle in TicketRepository, EmployeeRepository, TicketController.Details Controllers, UpdateTicket, views MongoIndexInitializer, employeeservice,ticketservice.createticket
Carlo	Dashboard: aggregaties voor status, percentages en werkverdeling, controller en view.	`ITicketQueries.cs`, `TicketQueries.cs`, `StatusPercentage.cs`, `EmployeeWorkload.cs`, `DashboardViewModel.cs`, `DashboardController.cs`, `Views/Dashboard/Index.cshtml`
Mart	Indexing en `MongoIndexInitializer`. filteren en sorteren van tickets (status, medewerker, datum). Uniek ticketnummer met retry. Opschonen van `ITicketRepository`. README; reviewen en mergen van pull requests.	`MongoIndexInitializer.cs`, `Program.cs`, `TicketRepository`, `ITicketRepository`, `ITicketService`, `TicketService`,`TicketController.Index`, `Views/Ticket/Index.cshtml`, `README.md`

Emre	Basis Tickets-module: datamodel, CRUD-operaties, repository en integratie met de webinterface.	`TicketController.cs`, `TicketService.cs`, `TicketRepository.cs`, `ITicketService.cs`, `UI Views (Ticket: Create, Edit, Index, etc.)`

7. Testen en uitvoeren

Testen

De CRUD-functionaliteit en dashboardqueries zijn gecontroleerd aan de hand van de MongoDB-database.

- **Tickets:** Het aanmaken, ophalen, bijwerken en verwijderen van tickets is gecontroleerd. Ook zijn filteren op status, filteren op toegewezen medewerker en sorteren op aanmaakdatum geïmplementeerd.
- **Medewerkers:** Het aanmaken, ophalen en bijwerken van medewerkers is geïmplementeerd. Daarnaast is gecontroleerd dat dubbele `employeeId`- en e-mailwaarden worden voorkomen. Medewerkers kunnen worden gedeactiveerd zonder hun gegevens uit de database te verwijderen.
- **Ticketdetails:** De reporter en assignee worden opgehaald met een MongoDB `$lookup` op de `employees`-collectie.
- **Dashboard:** De ticketstatuspercentages worden berekend met een MongoDB-aggregatiepipeline. De werkverdeling per medewerker wordt berekend met aggregaties op basis van `$group` en `$lookup`.
- **Indexen:** De indexen worden automatisch aangemaakt wanneer de applicatie start.

Configuratie en uitvoeren

Vereisten

- .NET 10 SDK.
- Toegang tot een MongoDB-database via MongoDB Atlas of een lokale MongoDB-instantie.

1. Configureer de databaseverbinding

De connectionstring staat niet in de repository. Vraag een teamlid om de connectionstring en stel deze lokaal in via .NET User Secrets.

Open een terminal in de map waarin `GardenGroup.csproj` staat en voer het volgende commando uit:

```
dotnet user-secrets set "ConnectionStrings:MongoDb"
"<your-mongodb-connection-string>"
```

Vervang `<your-mongodb-connection-string>` door de connectionstring van de database. Commit deze waarde nooit naar GitHub.

2. Start de applicatie

Voer vanuit de map met `GardenGroup.csproj` het volgende commando uit:

```
dotnet run
```

3. Database en indexen

De applicatie gebruikt de MongoDB-database `helpdesk` met de collecties `tickets` en `employees`. De benodigde indexen worden automatisch aangemaakt bij het opstarten van de applicatie.